

Name: Princess Beverly R. Ala-an
Section: BN01
Date: August 22, 2025

Reading Assignment:

Research the coding standards and best practices of your favorite programming language, and give your personal opinion on what could be improved in terms of readability and writability (e.g., changing printf to a clearer or shorter term).

Coding Standards and Best Practices in Python

Python follows a well-established set of coding standards known as PEP 8 (Python Enhancement Proposal 8). These standards aim to promote consistency across projects, improve readability, and make collaboration easier among developers.

Below are the key areas of Python's coding standards, along with my personal opinions on what could be improved.

The following mentioned after each point are the results of Python's Coding Standards.

1. Naming Conventions
 - Following of PEP's naming conventions
 - i. Wherein names that are visible to the user should reflect usage rather than implementation
 - ii. Naming styles should be descriptive and it should have consistent case styles
 - iii. It is discouraged to combine underscores and CamelCase for it becomes ugly

Python's naming conventions encourage clean and descriptive code, which greatly improves readability. However, I believe that naming conventions are only effective if programmers are disciplined enough to follow them. Without proper awareness, even Python's readability will not be achieved, and code may still end up looking complex and confusing.

2. Indentation and Formatting
 - Use of 4 spaces per indentation level
 - Avoid tabs for indentation
 - Align continuation lines either vertically (inside parentheses, brackets, or braces) or use hanging indents

I find Python's strict indentation rules both helpful and frustrating. On one hand, it enforces clean and structured code. On the other hand, compared to C, Python is less flexible since it does not use braces. Relying solely on indentation can cause confusion during debugging, as program logic is harder to trace. I believe Python could improve by allowing optional braces to give programmers more

flexibility.

3. Code Structure and Modularity

- Breaking long expressions over multiple line and use parentheses for readability.
- Usage of single space before and after operators
- Usage of Indentation rather than braces
- Each line is only one statement
- Functions (def) and classes (class) are used for encapsulation
- Code is organized into modules (.py files) and packages (folders with `__init__.py`).

Although PEP 8 provides strong guidelines, Python's library naming conventions can feel inconsistent. The use of `__init__.py` in packages makes the structure appear messy, especially for beginners. In contrast, C's library imports look cleaner and more straightforward. Python could benefit from simplifying its module and package system.

4. Error Handling

- Python has a lot of Error handling
 - i. Try-except blocks where the code inside try runs first. If an error occurs, Python jumps to the matching except.
 - ii. It can handle different exceptions such as `ValueError`, `TypeError`
 - iii. If the programmer doesn't know what error may occur, using generic `except` is convenient
 - iv. The `else` clause where the code inside runs if no error occurs
 - v. It catches specific exceptions and uses `finally` for cleanup
 - vi. This is Python's way of making programs robust and fault-tolerant, instead of crashing on the first error.

Python's error handling is one of its strongest features. Unlike languages that crash on the first error, Python makes it easy to catch and handle exceptions gracefully. This makes programs more robust and fault-tolerant. I have no major complaints here, as I find it both readable and practical.

5. Comments and Documentation

- Comments should be up to date when the code changes
- Comments should be complete sentences
- Comments are done using
 - i. `#` meaning block comment
 - ii. `""" """` meaning docstrings often used for documentation

While docstrings are Python's standard for documentation, I personally find them visually unappealing and sometimes distracting in the code. They often make the source look cluttered. A more lightweight or cleaner documentation style could improve the overall look of Python programs.

In conclusion, Python's coding standards—especially those defined in PEP 8—provide programmers with clear best practices that maximize readability and maintain consistency across projects. While Python excels at making code intuitive and developer-friendly, certain aspects such as strict indentation, package naming conventions, and documentation style could still be improved to offer greater flexibility and a cleaner aesthetic.